

LAB Assignment No 7

Interfacing DHT22 Sensor with ESP32 and Visualizing Temperature & Humidity on Node-RED Dashboard

Lab Description:

This lab demonstrates real-time environmental monitoring using IoT, where sensor data is transmitted via MQTT and visualized using a web dashboard.

Learning Objectives:

By the end of this lab, students will:

- Interface digital sensors (DHT22) with ESP32
- Publish real-time data using MQTT
- Visualize IoT data using Node-RED
- Build dashboards with charts and gauges
- Simulate IoT systems using Wokwi

Required Tools

Hardware (Simulated / Real)

- ESP32
- DHT22 Temperature & Humidity Sensor

Software

- Arduino IDE
- Node-RED
- MQTT Broker (e.g., [HiveMQ](#))

System Overview

DHT22 → ESP32 → MQTT → Node-RED Dashboard

Temperature Chart Humidity Gauge

Circuit Connections

DHT22:

Pin	ESP32
VCC	3.3V
GND	GND
DATA	GPIO 4

ESP32 Tasks

Students must:

1. Connect ESP32 to WiFi
2. Read **temperature & humidity** from DHT22
3. Publish data in JSON format:

 JSON

```
{  
  "temperature": 28.5,  
  "humidity": 65  
}
```

4. Use MQTT topic:

dht/data

ESP32 Code:

```
#include <WiFi.h>  
#include <PubSubClient.h>  
#include "DHTesp.h"  
  
const int DHT_PIN = 9;
```

```

DHTesp dhtSensor;

const char* ssid = "Wokwi-GUEST";
const char* password = "";
const char* mqtt_server = "broker.hivemq.com";

WiFiClient espClient;
PubSubClient client(espClient);
unsigned long lastMsg = 0;
#define MSG_BUFFER_SIZE (50)
float temp = 0;
float hum = 0;
int value = 0;

void setup_wifi() {

    delay(10);
    // We start by connecting to a WiFi network
    Serial.println();
    Serial.print("Connecting to ");
    Serial.println(ssid);

    WiFi.mode(WIFI_STA);
    WiFi.begin(ssid, password);

    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.print(".");
    }

    randomSeed(micros());

    Serial.println("");
    Serial.println("WiFi connected");
    Serial.println("IP address: ");
    Serial.println(WiFi.localIP());
}

void callback(char* topic, byte* payload, unsigned int length) {
    Serial.print("Message arrived [");
    Serial.print(topic);
    Serial.print("] ");
    for (int i = 0; i < length; i++) {
        Serial.print((char)payload[i]);
    }
    Serial.println();
}

```

```

// Switch on the LED if an 1 was received as first character
if ((char)payload[0] == '1') {
    digitalWrite(2, LOW); // Turn the LED on (Note that LOW is the voltage level
    // but actually the LED is on; this is because
    // it is active low on the ESP-01)
} else {
    digitalWrite(2, HIGH); // Turn the LED off by making the voltage HIGH
}

}

void reconnect() {
    // Loop until we're reconnected
    while (!client.connected()) {
        Serial.print("Attempting MQTT connection...");
        // Create a random client ID
        String clientId = "ESP8266Client-";
        clientId += String(random(0xffff), HEX);
        // Attempt to connect
        if (client.connect(clientId.c_str())) {
            Serial.println("Connected");
            // Once connected, publish an announcement...
            client.publish("IoT/mqtt", "IoT");
            // ... and resubscribe
            client.subscribe("IoT/mqtt");
        } else {
            Serial.print("failed, rc=");
            Serial.print(client.state());
            Serial.println(" try again in 5 seconds");
            // Wait 5 seconds before retrying
            delay(5000);
        }
    }
}

void setup() {
    pinMode(2, OUTPUT); // Initialize the BUILTIN_LED pin as an output
    Serial.begin(115200);
    setup_wifi();
    client.setServer(mqtt_server, 1883);
    client.setCallback(callback);
    dhtSensor.setup(DHT_PIN, DHTesp::DHT22);
}

void loop() {

```

```
if (!client.connected()) {
  reconnect();
}
client.loop();

unsigned long now = millis();
if (now - lastMsg > 2000) {
  lastMsg = now;
  TempAndHumidity data = dhtSensor.getTempAndHumidity();

  String temp = String(data.temperature, 2);
  Serial.print("Temperature: ");
  Serial.println(temp);
  client.publish("IoT/temperature", temp.c_str());

  String hum = String(data.humidity, 1);
  Serial.print("Humidity: ");
  Serial.println(hum);
  client.publish("IoT/humidity", hum.c_str());
}
}
```

Node-RED Tasks

MQTT Configuration

- Broker: `broker.hivemq.com`
- Port: `1883`

Flow Requirements

1. Receive Data

- MQTT IN node
- Topic:

`dht/data`

◆ 2. Convert JSON

- Use JSON node
-

◆ 3. Split Data (Function Node)

⌘ JavaScript

```
return [  
  { payload: msg.payload.temperature },  
  { payload: msg.payload.humidity }  
];
```

👉 Outputs: 2

◆ 4. Dashboard UI

📊 Temperature Chart

- UI Chart node
- Label: Temperature (°C)

📊 Humidity Gauge

- UI Gauge node
- Label: Humidity (%)

◆ 🔥 Final Flow

MQTT IN → JSON → Function (2 outputs)

✓
UI Chart

✓
UI Gauge

Dashboard URL

`http://127.0.0.1:1880/ui`

(or `/dashboard` for new dashboard)

Expected Output

- ✓ Temperature updates in real-time chart
- ✓ Humidity shown on gauge
- ✓ Data refresh every few seconds

Student Deliverables

1. Screenshots:
 - Wokwi simulation
 - Node-RED flow
 - Dashboard UI
2. Code:
 - ESP32 code
 - Node-RED flow JSON
3. Short explanation:
 - Data flow from sensor → dashboard

◆ ? Viva Questions

1. What is DHT22 and how does it work?
 2. Why JSON is used in IoT communication?
 3. Difference between chart and gauge?
 4. What happens if sensor returns NaN?
 5. Why MQTT is lightweight?
-

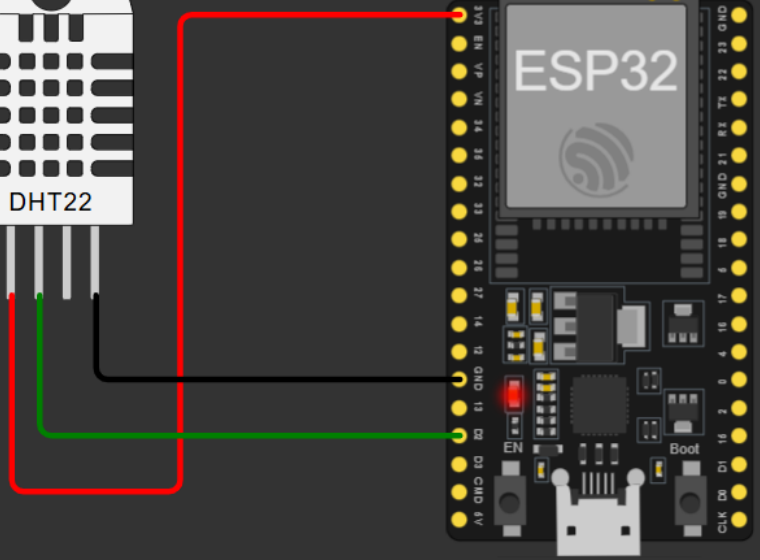
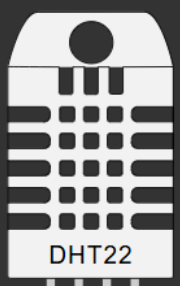
◆ ★ Bonus Tasks

- Add temperature alert (if $> 35^{\circ}\text{C}$)
- Store data in database
- Add historical graph (24 hrs)
- Combine with LED control

Simulation



00:24.379 18%



Humidity: 40.0

```
Connecting to Wokwi-GUEST
.....
WiFi connected
IP address:
10.10.0.2
Attempting MQTT connection...Connected
Temperature: nan
```



Attempting MQTT connection...Connected

Temperature: 24.00

Humidity: 40.0

Temperature: 24.00

Humidity: 40.0

Temperature: 24.00

Humidity: 40.0



```
1  #include <WiFi.h>
2  #include <PubSubClient.h>
3  #include "DHTesp.h"
4
5  const int DHT_PIN = 9;
6
7  DHTesp dhtSensor;
8
9  const char* ssid = "Wokwi-GUEST";
10 const char* password = "";
11 const char* mqtt_server = "broker.hivemq.com";
12
13 WiFiClient espClient;
14 PubSubClient client(espClient);
15 unsigned long lastMsg = 0;
16 #define MSG_BUFFER_SIZE (50)
17 float temp = 0;
18 float hum = 0;
19 int value = 0;
20
21 void setup_wifi() {
22
23     delay(10);
24     // We start by connecting to a WiFi network
25     Serial.println();
```

```

26     Serial.print("Connecting to ");
27     Serial.println(ssid);
28
29     WiFi.mode(WIFI_STA);
30     WiFi.begin(ssid, password);
31
32     while (WiFi.status() != WL_CONNECTED) {
33         delay(500);
34         Serial.print(".");
35     }
36
37     randomSeed(micros());
38
39     Serial.println("");
40     Serial.println("WiFi connected");
41     Serial.println("IP address: ");
42     Serial.println(WiFi.localIP());
43 }
44
45 void callback(char* topic, byte* payload, unsigned int length) {
46     Serial.print("Message arrived [");
47     Serial.print(topic);
48     Serial.print("] ");
49     for (int i = 0; i < length; i++) {

```

```

50         Serial.print((char)payload[i]);
51     }
52     Serial.println();
53
54     // Switch on the LED if an 1 was received as first character
55     if ((char)payload[0] == '1') {
56         digitalWrite(2, LOW); // Turn the LED on (Note that LOW is the voltage level
57         // but actually the LED is on; this is because
58         // it is active low on the ESP-01)
59     } else {
60         digitalWrite(2, HIGH); // Turn the LED off by making the voltage HIGH
61     }
62
63 }
64
65 void reconnect() {
66     // Loop until we're reconnected
67     while (!client.connected()) {
68         Serial.print("Attempting MQTT connection...");
69         // Create a random client ID
70         String clientId = "ESP8266Client-";
71         clientId += String(random(0xffff), HEX);
72         // Attempt to connect

```

```
72     // Attempt to connect
73     if (client.connect(clientId.c_str())) {
74         Serial.println("Connected");
75         // Once connected, publish an announcement...
76         client.publish("IoT/mqtt", "IoT");
77         // ... and resubscribe
78         client.subscribe("IoT/mqtt");
79     } else {
80         Serial.print("failed, rc=");
81         Serial.print(client.state());
82         Serial.println(" try again in 5 seconds");
83         // Wait 5 seconds before retrying
84         delay(5000);
85     }
86 }
87
88
89 void setup() {
90     pinMode(2, OUTPUT);    // Initialize the BUILTIN_LED pin as an output
91     Serial.begin(115200);
92     setup_wifi();
93     client.setServer(mqtt_server, 1883);
94     client.setCallback(callback);
```

```

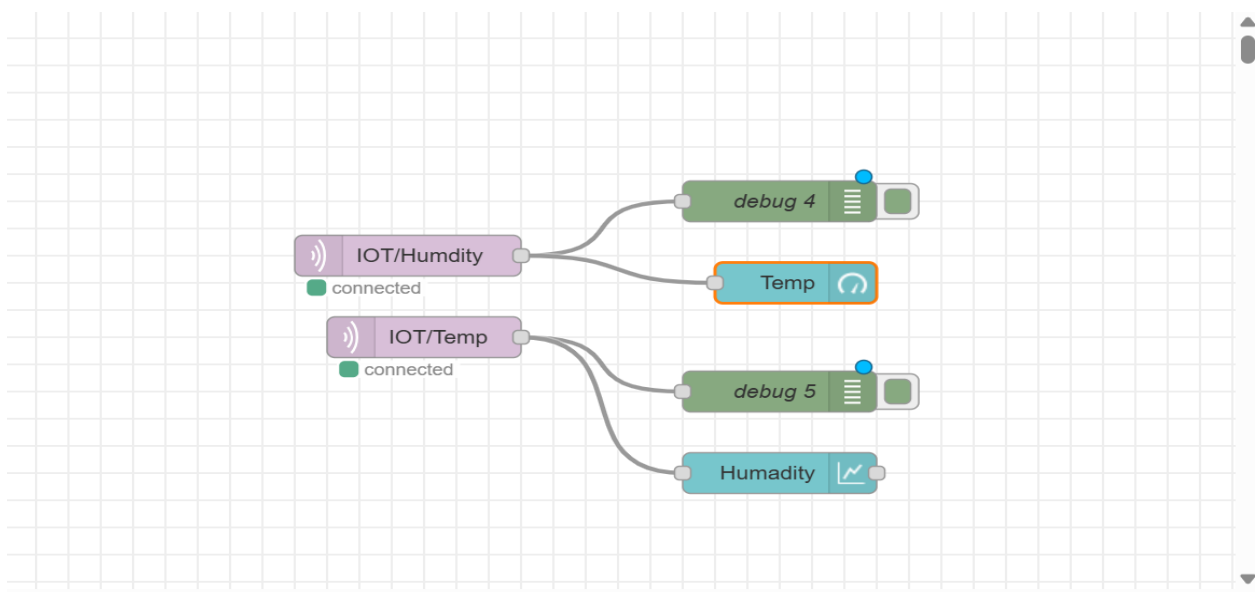
94     client.setCallback(callback);
95     dhtSensor.setup(DHT_PIN, DHTesp::DHT22);
96 }
97
98 void loop() {
99
100     if (!client.connected()) {
101         reconnect();
102     }
103     client.loop();
104
105     unsigned long now = millis();
106     if (now - lastMsg > 2000) {
107         lastMsg = now;
108         TempAndHumidity data = dhtSensor.getTempAndHumidity();
109
110         String temp = String(data.temperature, 2);
111         Serial.print("Temperature: ");
112         Serial.println(temp);
113         client.publish("IoT/temperature", temp.c_str());
114
115         String hum = String(data.humidity, 1);
116         Serial.print("Humidity: ");
117         Serial.println(hum);

```

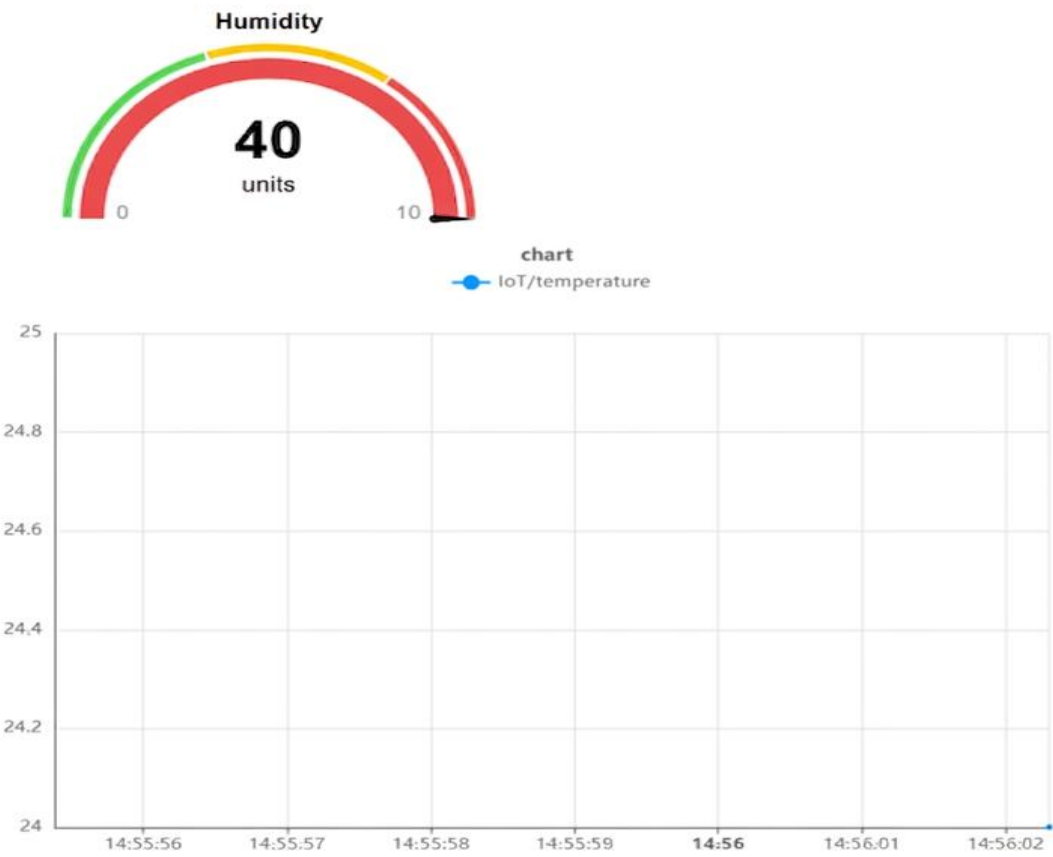
```

118         client.publish("IoT/humidity", hum.c_str());
119     }
120 }

```



TAB1



TAB1

